Morphic Studio Software Architecture Specification

Document 47 — AI Memory, Knowledge Graph & Context Management Architecture

Version: 1.0

Status: Approved

---

1. Purpose

This document defines how Morphic Studio stores, retrieves, organizes, and manages knowledge across the lifetime of a creative project.

Its objective is to provide AI with reliable, context-aware memory that preserves consistency while supporting long-term storytelling, collaboration, and production workflows.

---

2. Memory Philosophy

The memory system follows six principles.

2.1 Memory Before Intelligence

High-quality AI depends on high-quality memory.

The platform prioritizes accurate context retrieval before model generation.

2.2 Hybrid Memory Architecture

Memory is divided into three complementary layers.

Canonical Memory

Immutable project facts.

Stored in:

- Story Bible
- Canon Rules
- Character Profiles
- Timeline
- World Data

Only deliberate user actions modify canonical memory.

Creative Memory

Stores evolving creative intent.

Examples:

- Themes
- Writing style
- Tone
- Mood
- Symbolism
- Artistic direction
- Narrative goals

Creative Memory evolves throughout the project.

Working Memory

Short-lived operational context.

Examples:

- Current scene
- Recent edits
- Active conversation
- Current AI task
- Temporary notes

Working Memory expires automatically when no longer relevant.

2.3 Context over Conversation

AI retrieves project knowledge rather than relying on long chat histories.

Context should be assembled from structured project memory whenever possible.

2.4 Memory Is Searchable

Every memory object must be discoverable through semantic and structured search.

2.5 User Control

Users remain the authority over stored knowledge.

AI may recommend updates but never silently changes canonical memory.

2.6 Traceability

Every memory item records:

- Origin
- Creation time
- Last update
- Author
- Related entities
- Confidence (where applicable)

---

3. Knowledge Graph Architecture

The Knowledge Graph models relationships between entities.

Examples include:

Projects

↓

Characters

↓

Relationships

↓

Locations

↓

Organizations

↓

Artifacts

↓

Events

↓

Scenes

↓

Assets

↓

Productions

Each relationship includes metadata and directionality where appropriate.

---

4. Canonical Memory

Canonical Memory stores verified facts.

Examples:

Character age

Character abilities

Location descriptions

Timeline events

Magic rules

Technology rules

Governments

Historical events

Canon facts require explicit confirmation before modification.

---

5. Creative Memory

Creative Memory stores guidance rather than facts.

Examples:

Preferred pacing

Writing voice

Emotional goals

Recurring motifs

Audience expectations

Visual identity

Creative Memory influences generation without overriding canonical facts.

---

6. Working Memory

Working Memory maintains temporary context.

Includes:

Current editing session

Recent prompts

Recent AI outputs

Selected assets

Open documents

Temporary variables

Working Memory should remain lightweight and automatically expire.

---

7. Context Retrieval Pipeline

Every AI request follows:

Capability Request

↓

Intent Analysis

↓

Entity Detection

↓

Knowledge Graph Traversal

↓

Semantic Search

↓

Canonical Memory

↓

Creative Memory

↓

Working Memory

↓

Context Ranking

↓

Prompt Assembly

↓

AI Generation

Only relevant context should be included.

---

8. Retrieval-Augmented Generation (RAG)

The platform supports Retrieval-Augmented Generation.

Sources include:

Story Bible

Knowledge Graph

Creative Memory

Character Profiles

World Data

Production Notes

Assets

Project Documentation

Retrieved knowledge is injected into prompts before generation.

---

9. Timeline Management

Maintain chronological consistency.

Timeline tracks:

Historical events

Story events

Production milestones

Character development

Version history

Dependencies between events should be validated automatically.

---

10. Relationship Engine

Support relationships between:

Characters

Organizations

Locations

Scenes

Assets

Projects

Timeline events

Relationships should include:

Type

Strength

Direction

Status

History

---

11. Memory Synchronization

Changes to canonical entities automatically update dependent indexes and retrieval systems.

Synchronization should maintain consistency across:

Knowledge Graph

Search indexes

AI caches

Context retrieval

Analytics

---

12. Context Optimization

Prioritize context based on:

Current capability

Entity relevance

Recency

Canonical importance

User preferences

Token budget

The AI Gateway should optimize context to balance quality and cost.

---

13. Memory Lifecycle

Memory follows this lifecycle:

Create

↓

Validate

↓

Store

↓

Index

↓

Retrieve

↓

Update

↓

Version

↓

Archive

↓

Restore (if needed)

Every transition should be auditable.

---

14. Privacy & Security

Memory inherits platform security.

Requirements include:

Permission-aware retrieval

Encrypted storage

Audit logging

Role-based access

Project isolation

AI context filtering

No project memory may be exposed across project boundaries.

---

15. Acceptance Criteria

The memory architecture is successful if:

- Canonical facts remain consistent.
- Creative intent evolves without losing history.
- Working Memory remains efficient.
- Knowledge Graph relationships remain accurate.
- AI receives relevant context.
- Users retain authority over project knowledge.
- Long-term projects remain coherent.

Conclusion

The Morphic Studio memory architecture provides a structured, searchable, and context-aware foundation that enables AI to behave like a knowledgeable creative partner rather than a stateless assistant. By combining Canonical Memory, Creative Memory, Working Memory, a Knowledge Graph, and Retrieval-Augmented Generation, the platform preserves consistency, scales to long-running projects, and delivers AI assistance that remains aligned with the creator's vision over time.